

inTurno — Documentazione Funzionalità

inTurno è una web-app PWA multi-tenant per la gestione completa di turni, presenze e ordini di servizio di ristoranti multi-sede. Pensata per essere venduta come SaaS: ogni account manager gestisce i propri ristoranti in totale isolamento dagli altri.

URL produzione: <https://turni-nine.vercel.app>

Indice

- [1. Stack tecnologico e architettura](#)
- [2. Ruoli e permessi](#)
- [3. Onboarding SaaS: registrazione, demo, approvazione](#)
- [4. Area Manager — tab per tab](#)
- [5. Area Dipendente](#)
- [6. Area Consulente del Lavoro](#)
- [7. Bot Telegram con assistente AI](#)
- [8. Generazione turni con IA](#)
- [9. Notifiche push e PWA/offline](#)
- [10. Automazioni server](#)
- [11. Sicurezza](#)

1. Stack tecnologico e architettura

Livello	Tecnologia
Framework	Next.js 16 (App Router, Server Components, Server Actions, Turbopack)
UI	React 19 , Tailwind CSS , shadcn/ui (Radix), framer-motion , lucide-react
Backend	Supabase : Postgres, Auth, Row Level Security , Realtime (postgres_changes), Storage
Hosting	Vercel (deploy automatico da GitHub, branch <code>main</code>)
Lingua/fuso	Italiano, timezone Europe/Rome ovunque (<code>date-fns-tz</code>)
PWA	Service Worker custom (<code>public/sw.js</code>): cache offline, push, badge
Integrazioni	Bot Telegram con AI (Google Generative AI), Web Push (VAPID), email admin (Resend, opzionale)

Principi architetturali: - **Realtime ovunque:** turni, presenze, assenze, bacheca e approvazioni si aggiornano live senza ricaricare la pagina (subscription `postgres_changes`). - **Sicurezza server-side:** ogni mutazione passa da API route o Server Action che verifica ruolo, scope e stato account; la RLS del database è l'ultima linea di difesa. - **Multi-tenancy:** `profiles.managed_restaurant_ids` (`null` = platform owner che vede tutto il non-demo; array = ristoranti gestiti). I dati demo (`restaurants.is_demo`) sono invisibili agli account reali.

2. Ruoli e permessi

Ruolo	Visibilità	Poteri principali
	Globale su tutti i ristoranti non-demo	Tutto + approvazione nuovi account

Ruolo	Visibilità	Poteri principali
Manager (platform owner: <code>managed_restaurant_ids = null</code>)		
Manager (cliente SaaS: array di ristoranti)	Solo i propri ristoranti	Gestione completa di turni, staff, presenze, ODS, bacheca, report
Capo Servizio "Direttore" (<code>is_direttore = true</code>)	Il proprio ristorante, tutti i reparti	Come un manager ma confinato alla sede: crea/modifica turni, presenze, assenze, ODS; non può creare manager o consulenti
Capo Servizio	Il proprio ristorante, il proprio reparto	Turni e ODS del reparto; timbratura dal proprio dispositivo per i colleghi
Dipendente	Solo se stesso	Timbra con QR+GPS, vede i propri turni, completa ODS, legge la bacheca, richiede assenze
Consulente del Lavoro	Ristoranti assegnati (<code>consultant_restaurant_ids</code>)	Dashboard dedicata: report ore mensili (con o senza dettaglio orario in base a <code>can_view_hours</code>), scambio documenti/messaggi col manager

I reparti sono: **Sala, Pizzeria, Bar, Cucina**.

3. Onboarding SaaS

Registrazione (`/register`)

Un nuovo manager si registra con email reale e password. L'account nasce in stato **pending** e ottiene istantaneamente un **ambiente demo completo**: - **5 ristoranti demo** realistici, ognuno con **10 dipendenti** (account auth reali creati in parallelo) - ~2 mesi di **presenze storiche** (~2.400 timbrature), fasce orarie, 4 settimane di turni - **ODS** (generalisti, per reparto, per singolo dipendente) e **bacheca** popolata (comunicati globali, per ristorante, per reparto, personali)

Modalità demo (account pending)

- Banner arancione permanente in tutta l'app.
- **Sola lettura imposta lato server**: ogni endpoint di scrittura (`/api/users` , `/api/presenze` , `/api/assenze` , azioni turni/ODS/IA...) risponde 403 se `account_status = 'pending'` — nessun bypass possibile dal client.
- Isolamento totale: il demo vede **solo** i suoi 5 ristoranti demo; gli account reali non vedono mai dati demo (funzione RLS `can_manage_restaurant` esclude `is_demo` per il platform owner).

Approvazione (tab Account Pendenti, solo platform owner)

- Lista richieste con nome, email, data; pulsanti **Approva / Rifiuta** con spinner e rimozione ottimistica della card.
- **Approva**: cancella tutti i dati demo dell'utente, imposta `account_status = 'active'` con `managed_restaurant_ids = []` (ambiente vuoto, pronto per i ristoranti veri), invia email di conferma.
- **Rifiuta**: elimina account e dati demo.
- Badge numerico nella sidebar + notifica in-app + email all'admin a ogni nuova registrazione.

4. Area Manager

4.1 Dashboard

- KPI del giorno: presenti oggi, richieste pendenti, assenze.
- **QR Code del ristorante** (stampabile) per la timbratura dei dipendenti.
- **Timbrature fallback da approvare** (foto scattate quando il GPS non funziona) con anteprima immagine.
- **Preview presenze di oggi** per capo servizio/direttore, con aggiunta/modifica rapida (form unificato).

- Sezione **timbratura da dispositivo del capo servizio** (spezzato incluso).
- **Consulente del lavoro**: gestione consulenti collegati, inbox messaggi/documenti con realtime.
- Collegamento **bot Telegram** via deep-link.
- All'apertura esegue la **chiusura automatica dei turni dimenticati** (vedi §10).

4.2 Turni (la tab più ricca)

Griglia settimanale stile Excel — righe = dipendenti, colonne = Lun → Dom + **Monte Ore**: - Ogni cella mostra tutte le fasce del giorno (turni spezzati = più fascette) con colori: blu standard, ambra straordinario, rosso riposo. Click = modifica, "x" = elimina, "+" = crea per quella cella precompilata. - **Colonna "Monte Ore"**: totale ore stimate della settimana per dipendente, somma di tutte le fasce (spezzati inclusi) con gestione dei turni notturni che scavalcano la mezzanotte. - **Filtro ristorante** (manager) + **filtro reparto multi-selezione** (es. Sala + Bar insieme). - Navigazione settimana con frecce + "torna a oggi". - Aggiornamento **realtime**: un turno creato da un altro responsabile appare all'istante.

Form "Nuovo Turno" (un unico dialog, quattro modalità): 1. **Singolo**: dipendente, tipo (lavoro/riposo), data, orari con **durata calcolata live** (gestione notturni), straordinario, note. 2. **Turno spezzato nello stesso form**: "+ Aggiungi turno spezzato" aggiunge fasce extra per lo stesso giorno, ognuna con la propria durata; il totale si aggiorna; un solo Salva crea tutte le fasce. 3. **Inserimento Multiplo**: stesso orario ripetuto su un range di date, sui giorni della settimana selezionati (es. tutti i Lun-Ven di luglio). 4. **Più Giorni (orari diversi)**: la data diventa un compositore con frecce ◀ ▶; compili il turno di un giorno (spezzato incluso), passi al successivo, e i giorni compilati si accumulano in una lista "Giorni in programma" (click per rimodificare, x per togliere). Un solo Salva inserisce tutto in blocco.

Campi orario ovunque (`TimeInput`): digitazione libera da tastiera numerica con formattazione automatica HH:MM + icona orologio che apre comunque la rotella nativa.

Turni Fissi (pattern master): definisci per dipendente giorno-della-settimana + orario; "popola" genera i turni reali del periodo saltando i duplicati (chiave utente+data+ora, così i fissi spezzati generano entrambe le fasce).

Timeline Giornaliera (sotto la griglia, stile Gantt): - Ore in alto, nomi dipendenti a sinistra (colonna sticky), fasce posizionate proporzionalmente, linea rossa "adesso" che avanza da sola, navigazione giorno per giorno + click sulla data = **calendario nativo**, link "oggi" sempre visibile. - Scroll confinato al riquadro (orizzontale + verticale). - **Drag & drop (solo tablet/PC ≥768px)**: trascina i **bordi** (maniglie da 16px) per allungare/accorciare, trascina dal **centro** per spostare l'intero turno, trascina su **area vuota** per disegnare un turno nuovo. Snap a 15 minuti, salvataggio reale su DB (si riflette subito nella griglia sopra), anteprima ottimistica senza flash, selezione testo disabilitata durante il gesto. - **Su smartphone: sola visualizzazione** — nessun drag possibile, scroll sempre naturale, per evitare spostamenti accidentali.

4.3 Ristoranti

- CRUD ristoranti con **coordinate GPS** (per il geofencing delle timbrature) e **QR secret** rigenerabile.
- **Giorni di chiusura** settimanali.
- **Fasce orarie di servizio** per reparto (usate dalla generazione IA): orario, numero minimo di persone, indicatore turno notturno "(+1)", modifica in-place.

4.4 Dipendenti

- CRUD utenti con username (auth via email fittizia `@struttura.local`), ruolo, reparto, ristorante, permesso bacheca, flag direttore.
- Reset password dal manager.
- **Parametri per l'IA**: giorni di riposo settimanali, giorno di riposo preferito, ore settimanali target (part-time), fasce principali (checkbox sulle fasce reali del reparto), **fasce jolly** su altri reparti con priorità, può sostituire il capo servizio, anzianità.
- Consulenti del lavoro: ristoranti assegnati + flag "può vedere gli orari".

4.5 Presenze

- Elenco mensile raggruppato per giorno → per dipendente: blocchi orari cliccabili (spezzato = più blocchi con badge "Spezzato"), badge "In corso" con **contatore live**, totale ore per giorno, ristorante.
- Dipendenti **assenti** mostrati nello stesso elenco con il tipo di assenza (ferie/malattia/riposo/ingiustificata).
- Filtri mese + ristorante; indicatore "Live" della connessione realtime.
- **Form unificato Aggiungi/Modifica** (stessi 4 campi: dipendente — precompilato e bloccato in modifica —, data, ora ingresso, ora uscita) con eliminazione a doppia conferma.
- **Turni notturni**: uscita "prima" dell'ingresso (es. 15:00 → 01:00) viene registrata automaticamente al giorno successivo.
- Sezione approvazione timbrature fallback (foto).

4.6 Assenze

- CRUD assenze approvate (ferie, malattia con codice certificato INPS, riposo, assenza ingiustificata) su range di date, filtri mese/ristorante, **realtime** sulle richieste in arrivo.

4.7 Approvazioni

- Richieste di assenza dei dipendenti in attesa: approva/rifiuta con realtime.

4.8 Bacheca

- Comunicati con **targeting**: tutti, per ruolo, per ristorante, per reparto, per dipendenti specifici (multi-selezione).
- **Conferme di lettura**: contatore "letto da N persone" + elenco nominativo con orario.
- **Notifica push** automatica ai destinatari alla pubblicazione.
- **Realtime**: comunicati di altri responsabili appaiono/spariscono live.
- Il direttore pubblica nel proprio ristorante (tutto/reparto/singolo); il capo servizio abilitato può pubblicare.

4.9 ODS — Ordini di Servizio

- Compiti per reparto o assegnati a un singolo dipendente, con ricorrenza: **quotidiana, settimanale (1 giorno), bisettimanale (2 giorni), straordinaria**.
- Reset alle **4:00 di notte** (cutoff): ogni giorno i quotidiani ripartono da zero.
- Vista manager: filtri ristorante/reparto, spunta di completamento con "completato da N persone" ed elenco nominativo, evidenza dei task non previsti oggi.
- Notifiche al manager quando i dipendenti completano.

4.10 Report

- **Tabella mensile stile foglio ore**: righe = dipendenti, colonne = giorni del mese; celle con ore lavorate/assenze (P/F/M/R...), totali riga e colonna.
- **Modifica per cella**: click su una cella apre l'editor del giorno (più fasce ingresso/uscita con aggiungi/rimuovi, oppure assenza con tipo/certificato/note) — crea, modifica ed elimina passando dalle stesse API validate (turni notturni gestiti).
- Filtri per ristorante/dipendenti; esclusione automatica dei turni aperti.
- **Esportazioni**: Excel/CSV e **PDF unificato** per il consulente del lavoro.
- All'apertura esegue la chiusura automatica dei turni dimenticati.

4.11 Account Pendenti

Vedi §3 — approvazione/rifiuto registrazioni con feedback immediato.

5. Area Dipendente

Interfaccia mobile-first con **bottom nav** (Home / ODS / Turni) e badge non-letti.

Home — Timbratura

- Orologio live + pulsante **Scansiona Ingresso/Uscita**.
- Flusso: verifica **GPS** (geofence 100m + tolleranza precisione fino a 250m per segnale debole indoor) → scanner **QR** → registrazione. Fail-closed: senza GPS niente timbratura "normale".
- **Fallback foto**: se il GPS nega o sei "troppo lontano" ma sei nel locale, timbri scattando una foto (compressa client-side ~200KB) che finisce in approvazione al manager.
- **Turno spezzato automatico**: una seconda timbratura nello stesso giorno viene marcata `is_split_shift` e notificata al manager.
- **Offline-first**: senza rete la timbratura viene salvata in coda IndexedDB con il timestamp reale "congelato" e sincronizzata appena torna la connessione. Vale anche per le spunte ODS.
- Contatore "Turno in corso" live; richiesta assenza (ferie/malattia/...) con date e note; bacheca a drawer con segnalibro non-letti.

I Miei Turni

- Prossimi e passati, **raggruppati per giornata**: un turno spezzato appare come **una sola riga** con tutte le fasce ("11:00 – 16:00 / 20:00 – 23:59") e tag "Spezzato"; badge Standard/Straordinario/Riposo; note.
- **Realtime**: il turno assegnato dal manager appare all'istante; refetch al mount per essere immuni da cache.

ODS

- Istruzioni personali + di reparto con filtri per tipo, spunta con animazione, aggiornamento ottimistico e coda offline. Le settimanali/bisettimanali appaiono solo nei giorni previsti.

6. Area Consulente del Lavoro

- **Dashboard dedicata** (`/consulente/dashboard`) con i soli ristoranti assegnati.
- **Report ore mensile** per ristorante: totali per dipendente; il dettaglio dei singoli orari è visibile solo se il manager ha concesso `can_view_hours`.
- **Inbox bidirezionale** col manager: messaggi + upload/download documenti (buste paga, comunicazioni), stato letto/non letto, realtime.
- Download **PDF unificato** del mese.

7. Bot Telegram

Collegamento sicuro via deep-link dalla dashboard (token monouso legato al profilo). Rispetta ruolo e scope dell'utente collegato.

Comandi: `/start`, `/help`, `/turni`, `/nuovoturno` (wizard guidato), `/riposo`, `/eliminaturno`, `/ods`, `/nuovoods`, `/completaods`, `/presenze`, `/nuovapresenza`, `/modificapresenza`.

Assistente AI: qualsiasi messaggio libero (senza comando) viene interpretato dall'IA (Google Generative AI) con contesto e cronologia: puoi chiedere "chi lavora domani sera?", "metti Paolo di riposo giovedì", "quante ore ha fatto Giorgia questa settimana?" e il bot esegue o risponde sui dati reali del tuo scope.

8. Generazione turni con IA

Pulsante **"Genera con IA"** nella tab Turni (per ristorante selezionato): 1. **Wizard**: settimana target, reparti, chiusure straordinarie, note in linguaggio naturale. 2. **Apprendimento dallo storico presenze**: per ogni dipendente rileva pattern reali — chi fa lo spezzato (e con che pausa media), orari medi mattina/sera, "zone morte" con personale ridotto. 3. **Algoritmo deterministico**: rispetta fasce orarie del ristorante e coperture minime, giorni di riposo (preferendo i feriali), ore target dei part-time, almeno un senior per fascia dove richiesto, **jolly cross-reparto** per coprire i buchi, proposte di straordinario segnalate. 4. **Bozza interattiva**: griglia settimanale con modifica per cella (orari con `TimeInput`), rifiuta/ripristina singoli turni, pannello avvisi (coperture scoperte, vincoli violati). 5. **Conferma**: inserimento massivo dei turni reali + notifica push ai dipendenti coinvolti. Oppure scarta tutto.

9. Notifiche push e PWA

- **Web Push VAPID** con banner di attivazione: nuovi comunicati in bacheca, turni confermati dall'IA, completamenti ODS, nuove registrazioni (admin).
- **Service Worker**: pre-cache dell'app shell, **NetworkFirst per pagine e navigazioni RSC** (mai dati vecchi da cache), CacheFirst solo per asset immutabili; **auto-reload alla pubblicazione di una nuova versione** (l'app si aggiorna da sola dopo ogni deploy); App Badge; installabile su Home (manifest).
- Coda offline IndexedDB per timbrature e ODS (vedi §5).

10. Automazioni server

- **Chiusura turni dimenticati:** un job `pg_cron` (`close-stale-shifts` , ogni 30 minuti, direttamente nel database) chiude le timbrature aperte da più di 16 ore fissando l'uscita a **ingresso + 7 ore**. Sweep aggiuntivi all'apertura di Dashboard, Presenze, Report e a ogni timbratura, per feedback immediato.
 - **Popolamento da turni fissi** con anti-duplicazione per utente+data+ora inizio.
 - **Cutoff ODS alle 4:00** per il reset dei task quotidiani.
-

11. Sicurezza

- **RLS Postgres** su tutte le tabelle con funzioni `SECURITY DEFINER` (`get_my_role()` , `get_my_managed_restaurant_ids()` , `can_manage_restaurant()` , `is_demo_restaurant()`) per evitare ricorsioni e garantire l'isolamento multi-tenant e demo/reale.
 - **Doppio controllo:** ogni API route/Server Action ri-verifica ruolo, scope (ristorante/reparto del chiamante e del bersaglio) e `account_status` prima di toccare i dati; la RLS blocca comunque payload contraffatti.
 - **Geofence server-side** sulla timbratura (Haversine, non solo client).
 - QR secret per ristorante, rigenerabile; token Telegram monouso; nessun segreto nel codice client (solo chiavi `NEXT_PUBLIC_*`).
-

Documento generato dal codice sorgente del repository `thebearpizza/Turni` . Per il codice completo vedere `CODICE_COMPLETO.md` .